



Numba v2: Towards a SuperOptimizing Python Compiler

Siu Kwan Lam



Disclaimer

- This talk is about new version of Numba that is **not yet available**
- Current version of Numba **continues to be fully supported**
 - Many new releases are planned
 - Stability is a very high priority
- New components of v2 will be added to existing version when appropriate

Numba v1 optimizes Python code
by translating bytecode to LLVM

What can we do that LLVM cannot?

Finding the right optimization is hard

What if the compiler **explores all possible optimizations**

Evaluating code performance is hard work

What if the compiler **finds the best optimization**
for you?

Numba v2 is a **SuperOptimizer**

- Explore all possible program rewrites
- Extract the best program

For example

Rewrite

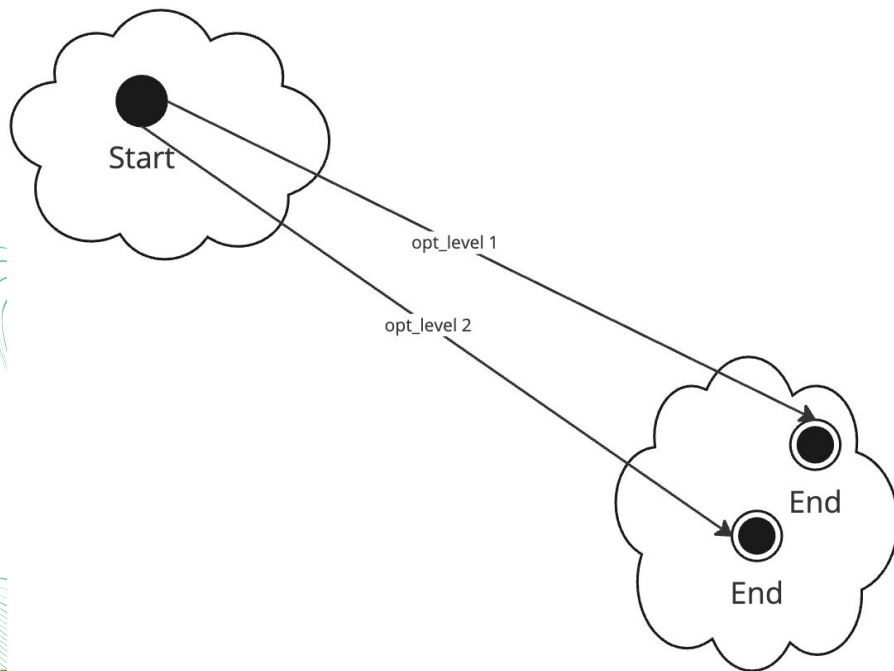
`(MatrixA @ MatrixB) @ MatrixC`

To

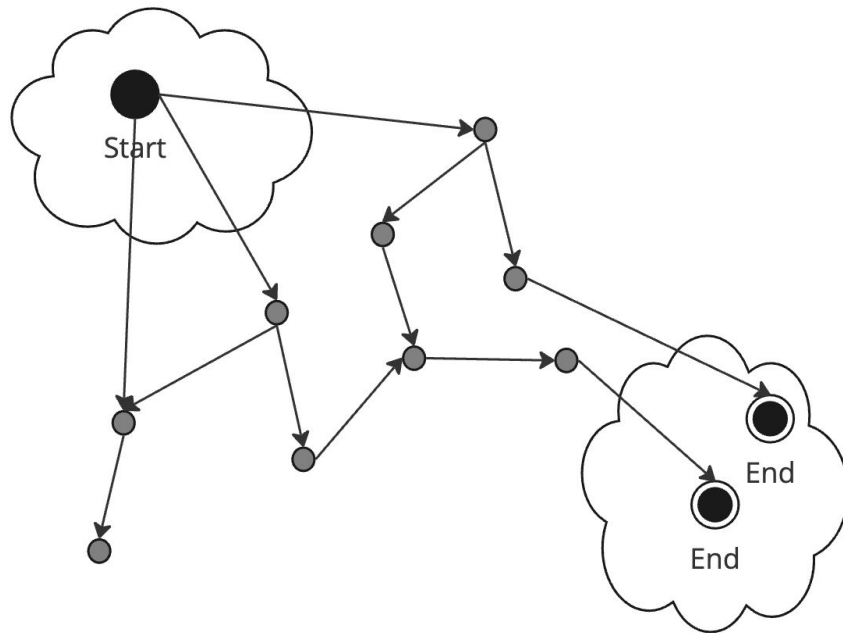
`MatrixA @ (MatrixB @ MatrixC)`

SuperOptimization with Equality Saturation

Traditional (Heuristic) Compiler



Equality Saturation Compiler

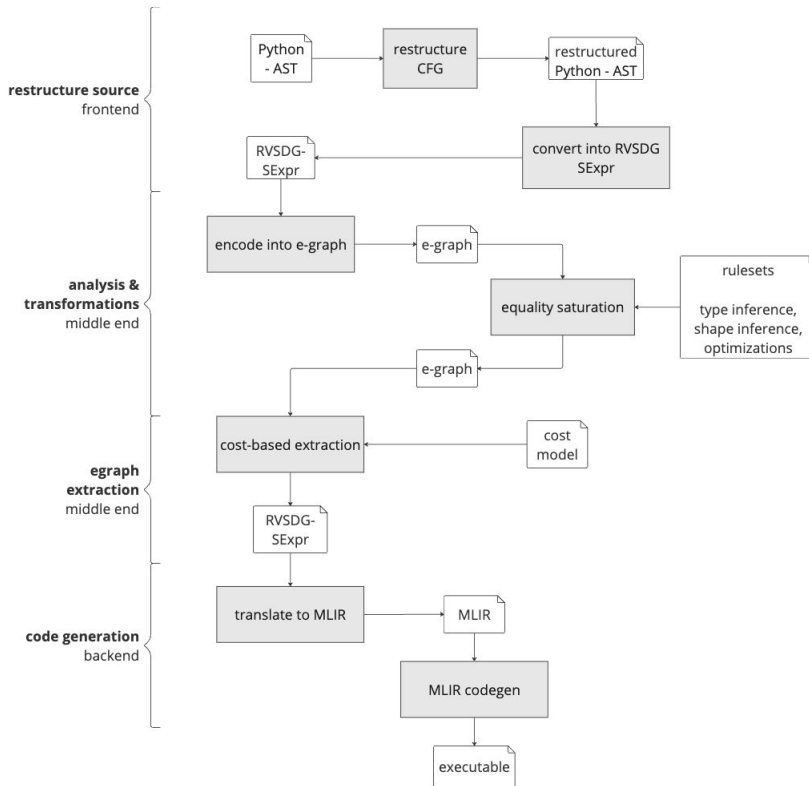


Compiler Pipeline

AST frontend

Analysis &
SuperOptimization
in EGraph

Get optimized code
(Demos will use
extracted Python
source)



Demo 1:

Associativity of Matrix Multiplication

A Better way to optimize code

Declarative and composable rules like this:

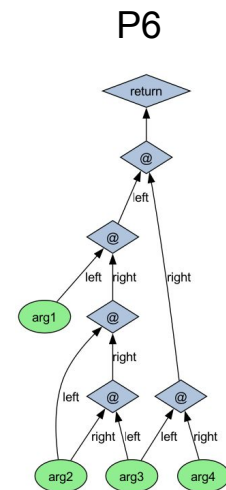
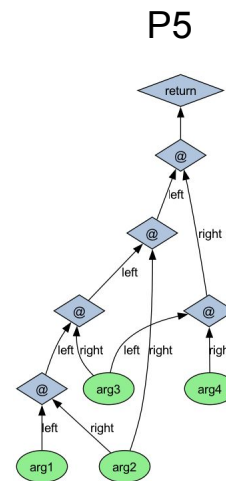
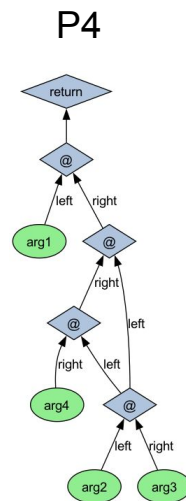
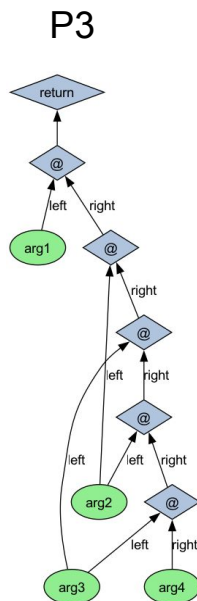
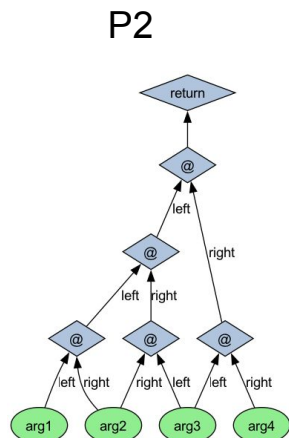
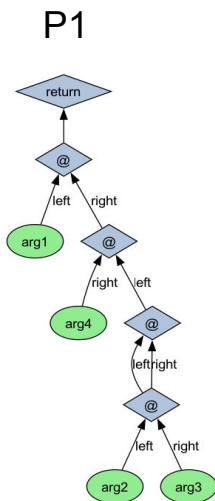
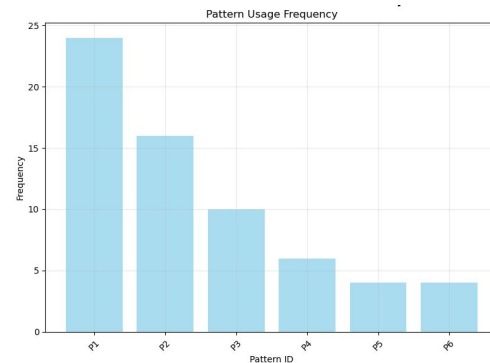
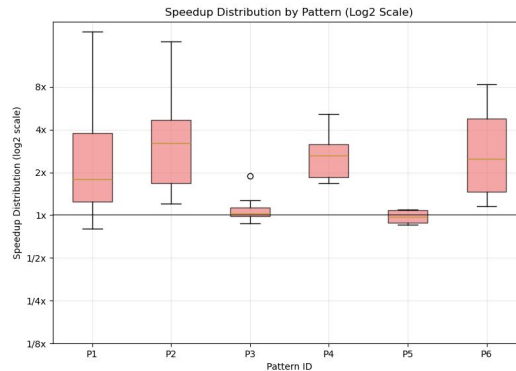
```
rewrite(MatMul(MatMul(A, B), C)).to(  
    MatMul(A, MatMul(B, C))  
)
```

Instead of pass-ordering hell (phase ordering problem),

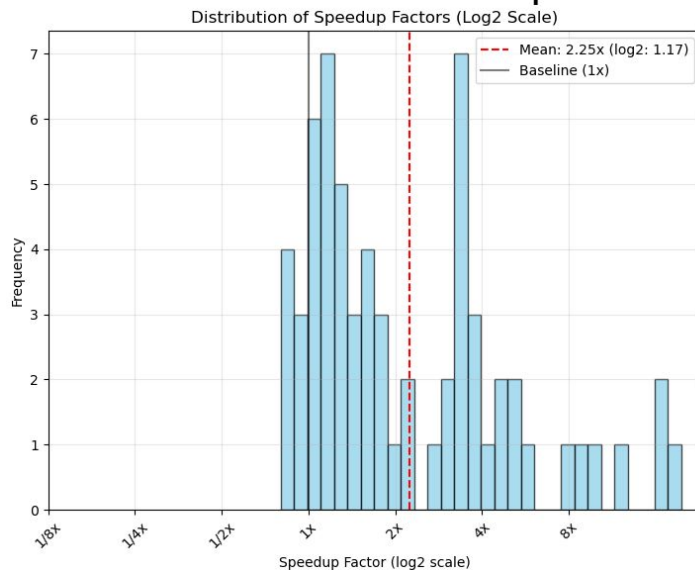
The rule runs repeatedly and recursively until *saturation* (cannot apply anymore)

Evaluation

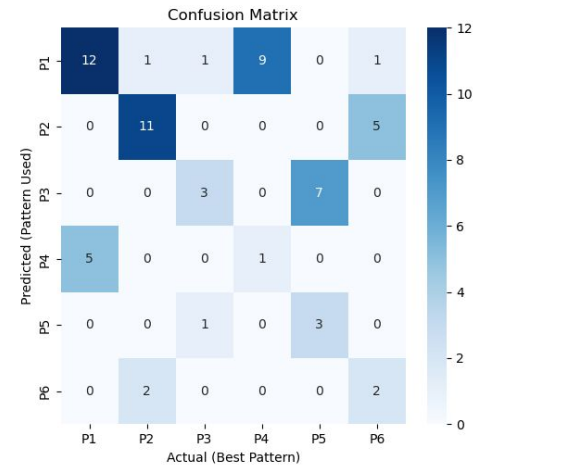
- Small grid search over matrix sizes
- 6 variants



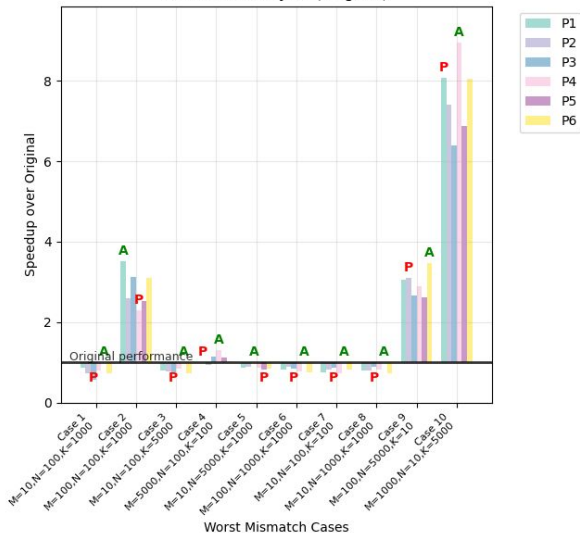
Total benchmarks analyzed: 64
 Average speedup: 3.37x
 Median speedup: 1.70x
 Max speedup: 19.67x
 Min speedup: 0.80x
 Standard deviation: 3.95



- Despite overly simplistic cost model
- Extracted versions are mostly beneficial
 - Median improvement is 2.25x



Speedup Comparison: Worst Mismatches
 (P=Predicted, A=Actual Best)
 Bars rooted at y=1 (Original)



Demo 2:

Tensor Algebra Rules from TASO/TENSAT



TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions

Zhihao Jia
Stanford University
zhiahao@cs.stanford.edu

Oded Padon
Stanford University
padon@cs.stanford.edu

James Thomas
Stanford University
jjthomas@stanford.edu

Todd Warszawski
Stanford University
twarszaw@stanford.edu

Matei Zaharia
Stanford University
matei@cs.stanford.edu

Alex Aiken
Stanford University
aiken@cs.stanford.edu

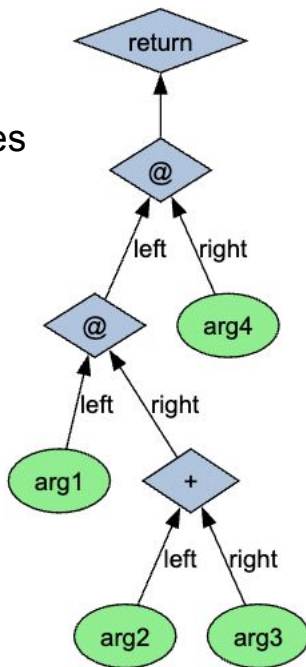
EQUALITY SATURATION FOR TENSOR GRAPH SUPEROPTIMIZATION

Yichen Yang^{*1} Phitchaya Mangpo Phothilimthana² Yisu Remy Wang³ Max Willsey^{3,4} Sudip Roy²
Jacques Pienaar²

TASO/TENSAT Example

```
def original_mma(input_1, input_2, input_3, input_4):
    return input_1 @ (input_2 + input_3) @ input_4
```

- Combining associativity with new rules (on the right)
 - TASO/TENSAT rules
 - Distributive rules

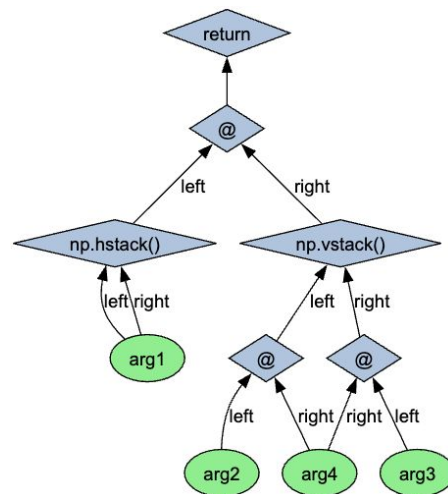
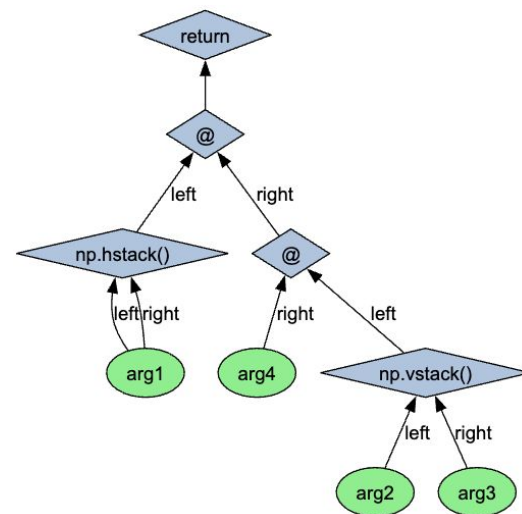
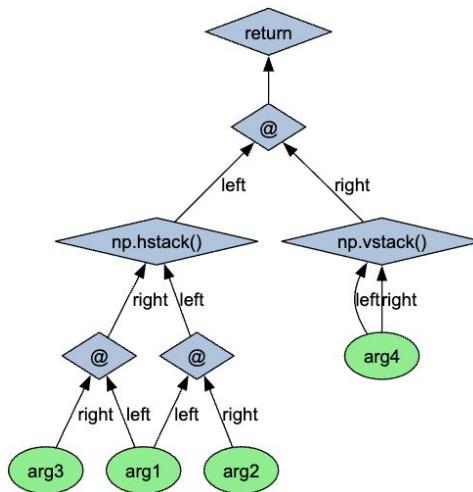


```
# tensat rule
rewrite(
    ewadd(matmul(ary1, ary3), matmul(ary2, ary4)),
).to(matmul(hstack(ary1, ary2), vstack(ary3, ary4)))

# distributive
rewrite(
    matmul(ary1, ewadd(ary2, ary3))
).to(
    ewadd(matmul(ary1, ary2), matmul(ary1, ary3)),
)
rewrite(
    matmul(ewadd(ary2, ary3), ary1)
).to(
    ewadd(matmul(ary2, ary1), matmul(ary3, ary1)),
)
```

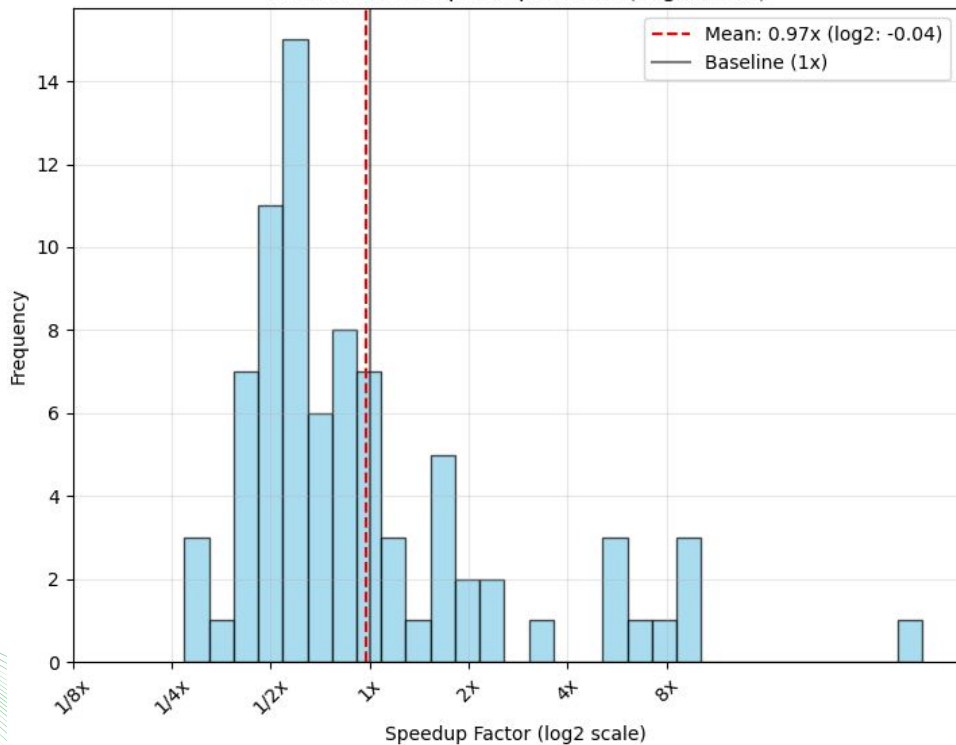
Evaluation

1. Distributive: $A @ (B + C) \rightarrow A @ B + A @ C$
2. Distributive: $(A + B) @ C \rightarrow A @ C + B @ C$
3. Associativity: $(A @ B) @ C \rightarrow A @ (B @ C)$
4. TenSat: $A @ C + B @ D \rightarrow \text{hstack}(A, B) @ \text{vstack}(C, D)$
5. Factoring: $\text{vstack}(A @ C, B @ C) \rightarrow \text{vstack}(A, B) @ C$



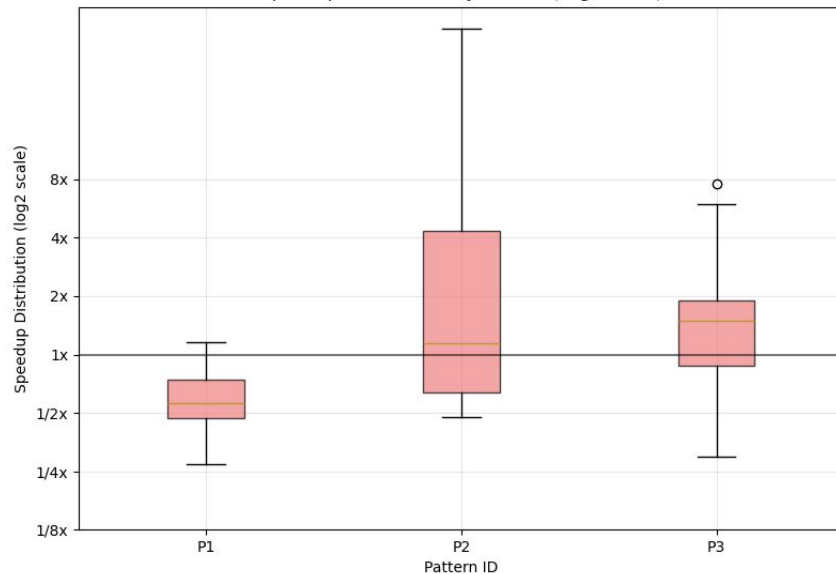
Evaluation

Distribution of Speedup Factors (Log2 Scale)



Most cases are slower indicating problem with the **overly simplistic cost-model**

Speedup Distribution by Pattern (Log2 Scale)



Technical Details

Numba v2 is needed because...

(BTW, we have 10+ years of experience of building Numba)

Phase ordering problem

Python highly dynamic semantic needs highly dynamic pass ordering

Solution: equality saturation

Difficulty for domain experts to add optimizations

Difficult to add passes to custom data types/classes

Solution: declarative & composable rules

Tensor optimization in general-purpose language

Solution: Higher-level program representation, E-graph analysis, rewrite rules



Egraph? SuperOptimization?

Each hull is an e-class of equivalent terms

Edges represent value dependence

- Use **EGraph + Equality Saturation** to find all **observationally equivalent** variants of every program term
- Select the optimal solution
 - **Extract by minimizing cost**
 - Iterative relaxation algorithm

Equality Saturation

E.g. E-class
(left-associative ==
right-associative)

Egraph Extraction

E-Graph Input

EClass A (Root)

f(B, C)

Cost: 1.0 + child_costs

optimized_f(B, C)

Cost: 0.8 + child_costs

EClass B

x + y

Cost: 1.5

add(x, y)

Cost: 2.0

EClass C

z * 2

Cost: 1.0

z << 1

Cost: 0.5

mul(z, 2)

Cost: 1.8

EClass C

z * 2

Cost: 1.0

z << 1

Cost: 0.5

mul(z, 2)

Cost: 1.8

Before Selection

EClass A Bucket

f(B, C)

Total: 3.0

optimized_f(B, C)

Total: 2.8

EClass B Bucket

x + y

Total: 1.5

add(x, y)

Total: 2.0

EClass C Bucket

z * 2

Total: 1.0

z << 1

Total: 0.5

mul(z, 2)

Total: 1.8

After Selection

EClass A Winner

f(B, C)

Total: 3.0

optimized_f(B, C)

✓ Best: 2.8

EClass B Winner

x + y

✓ Best: 1.5

add(x, y)

Total: 2.0

EClass C Winner

z * 2

Total: 1.0

z << 1

✓ Best: 0.5

mul(z, 2)

Total: 1.8

Where do we use Equality Saturation?

E-graph and Equality Saturation are very POWERFUL

They come from SAT solvers and automatic theorem provers.

We use it for

- Type inference
- Shape inference
- Constant propagation and folding
- Loop analysis
- Optimization rewrites

The only essential “pass”

Saturation

Risks

- Use up all memory

Derisk:

- Don't saturate; limit by time budget
- type inference is the only essential "pass" & does not grow the e-graph exponentially.

Extraction

Risks:

- True optimal solution is **NP-hard**

Derisk:

- Iterative relaxation to approximate. May not give true optimal result.
- Limit optimization by time budget.

Future

Better Cost Model

- Currently uses simple FLOP estimation
- Future work includes looking at N-d vector cost
 - Memory cost (non-local memory/cache effect)
 - Compute cost
 - Energy cost
 - Floating point accuracy (is fast-math allowed or strict IEEE-754?)
- e.g. machine learning on profile data

How to enable the community to contribute optimization rules?

- Imagine a future where you can download optimization rules from experts
- Declarative rule from egglog is not the most intuitive API for Python programmers

Why Math Rewrites Matters in Compilers

Compilers can generally perform fusion.
However, **optimizations that require mathematical rewriting of the same expression** (while maintaining numerical stability) are generally **harder for compilers**.

Dao 2024 [answering why compiler cannot create FlashAttention-2](#) ([link](#))

Dao 2024 @doi:10.48550/arXiv.2307.08691

Our next steps

- Implement LLaMa-3 in Python+NumPy using Numba v2
- Evaluate:
 - Auto-optimization
 - Is it possible to derive flash-attention from rewrite rules?
 - Can it emerge from non-specific rules?
 - Scaling to GPU
 - Practicality



Q & A

Thank you for your time!

FAQ

Q: When is it available?

- Not yet. See tutorial for now.

Q: Does it support control-flow?

- Yes. Example loop pass [example](#).
Example program rewrite [ch03 of the tutorial](#).

Q: Does it support GPU?

- Yes, currently we have automatic offload via MLIR Affine dialect. See [ch08 of the tutorial](#).



To learn more, see our scipy proceeding submission and **Kaleidoscope-style tutorial**:

https://numba.pydata.org/numba-prototypes/sealir_tutorials/index.html